

LIGHTGBM-BASED HULL TACTICAL-MARKET PREDICTION COMPETITION

Authors: Jia Ru (123090221, L01)
Xiao Yue (123090660, L02)
Zhu Kexin (123090921, L02)
Zhong Chenhao (123090893, L01)
Chen Xiang (123090040, L01)

ABSTRACT

Our project builds a compact pipeline for processing financial time-series data and training a LightGBM model for short-term market prediction. The resulting strategy delivers stable signals with controlled volatility, highlighting both the potential and limitations of machine-learning approaches in market timing.

1 INTRODUCTION

Financial markets are widely believed to follow the Efficient Market Hypothesis (EMH), which claims that publicly available information is already reflected in asset prices, making it extremely challenging to generate persistent excess returns. However, recent advances in data-driven modeling and the availability of rich macroeconomic, technical, and sentiment features provide new opportunities to revisit the problem of market predictability. The Hull Tactical – Market Prediction competition on Kaggle offers a structured environment to explore this question by asking participants to predict the S&P 500’s daily excess return and determine the optimal portfolio exposure between 0 and 2 for each trading day.

Unlike traditional return forecasting tasks, this competition evaluates participants using a volatility-adjusted Sharpe ratio that penalizes both excessive risk-taking and bad performance relative to the market benchmark. Thus, the objective is not only to predict short-term market movements, but also to construct a stable trading strategy whose risk profile remains close to the underlying index. This evaluation setting encourages models that balance predictive accuracy with practical portfolio considerations such as leverage, volatility, and return stability.

The dataset provided for the competition spans six months of historical market data and contains lots of heterogeneous features, including macroeconomic indicators, interest rate variables, valuation metrics, volatility measures, momentum factors, and sentiment signals. These high-dimensional and noisy financial time series create challenges related to missing values, nonstationarity, and feature redundancy, while also offering opportunities for effective feature engineering and model design.

In this project, we build a complete pipeline for financial prediction and portfolio weighting under competition constraints. The work includes dataset exploration, feature preprocessing, model construction, validation using the competition’s official evaluation API, and generation of final predictions for the Kaggle leaderboard. Through this process, the project aims to examine whether machine-learning-based signals can improve risk-adjusted returns and provide information on the complexities of financial forecasting in real-world settings.

2 DATA EXPLORATION

Before modeling, a systematic exploratory data analysis pipeline was conducted to obtain a comprehensive understanding of the hull tactical market prediction dataset, which is a standard and necessary preprocessing step in machine learning. The data structure and quality are essential for the subsequent model selection and training.

2.1 DATASET PROCESS

The basic characteristics of the dataset, including the number of rows and columns, as well as the total memory usage, were summarized. The shape of the training dataset is (9021, 98), and its total memory usage is 6.74 MB. These features are important for high-dimensional financial time-series data, as they reflect the dataset scale and the required computational resources.

Missing values were assessed by calculating the absolute counts and percentages for all columns, followed by visualizations of the top 20 features with the highest missing rates. Duplicate rows were checked to verify data integrity. Results showed that 85 columns contained missing values (including 8 with over 50% missing rates), while no duplicate rows were detected.

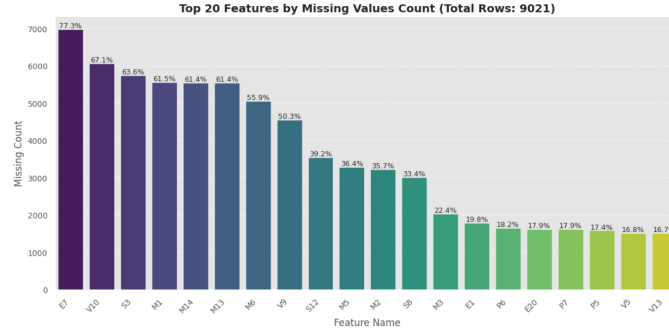


Figure 1: Top 20 Features by Missing Values Count

Outliers were detected using the Interquartile Range (IQR) method, which is widely used to identify extreme values in continuous distributions Information & Sciences (2018). For each column, values outside of $[Q1 - 2.5 \times IQR, Q3 + 2.5 \times IQR]$ were flagged as outliers. A total of 43 columns contained outliers, with their outlier counts and percentages summarized.

2.2 FEATURE SELECTION

We first performed a correlation analysis on the features. However, the results showed that the correlation between the features was not high. The figure below shows the correlation between a randomly selected subset of features (in the actual model training, we performed a correlation analysis on all features).

Furthermore, given the complicated nature of financial time-series data, features selected by traditional importance ranking showed limited correlation with target variables. Therefore, we implemented an intelligent random forest model to identify optimal feature combinations through performance assessment.

Based on the data exploration part indicating quality issues, the training data was preprocessed. The first 1005 rows were excluded because of excessive missing values. For the remaining dataset, a two-stage imputation strategy was implemented. The remaining missing values were first forward filled by adjacent values, followed by any still remaining missing values being replaced by 0. This preprocessing procedure resulted in a clean and temporally consistent training dataset suitable for feature analysis.

All available columns, excluding non-predictive variables (date_id, forward_returns, risk_free_rate, and market_forward_excess_returns), constituted the potential feature pool. The training dataset was then chronologically split into training (80%) and validation (20%) datasets, preserving temporal dependencies. Next, a random forest regressor served as the evaluation model with mean squared error (MSE) as the performance evaluation criterion. 60 independent trials were conducted, each trial randomly selecting 30 features from the 94 candidate features. The feature combination with the lowest MSE was selected for subsequent feature engineering and model training. Since we have set the seed as 42, so the selected features are same at each tempts:

'E13', 'I4', 'P13', 'V1', 'P1', 'S12', 'E10', 'M9', 'M12', 'M18', 'I1', 'M17', 'E20', 'E1', 'S10', 'E14', 'V3', 'S6', 'E5', 'M2', 'S11', 'E18', 'I3', 'D7', 'S7', 'E16', 'E12', 'E11', 'M11', 'P7'

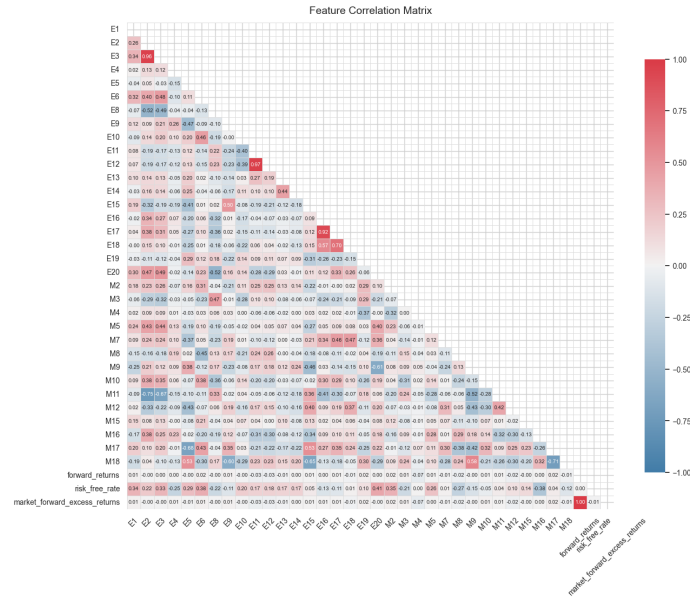


Figure 2: Feature correlation matrix

2.3 FEATURE ENGINEERING

Following the data exploration and feature selection parts, feature engineering was performed to enhance the model's ability to capture the temporal and non-linear relationships in financial time-series data with a limited number of features.

To capture delayed market responses and the underlying correlated patterns of the training data, temporal lag variables were created. Specifically, for each selected feature, the lagged periods were [1, 3, 5, 7, 14, 20], which enabled the model to access historical information across short, medium, and long-term effects. Additionally, we further engineered differencing features with 1-order and 7-order, which had a similar effect.

Rolling window statistics, including means, standard deviations, optima and medians, were computed across five window sizes [3, 6, 10, 20, 60], which summarized the evolving market dynamics. Additionally, we further engineered percentage change (momentum) features over [1, 3, 7] periods and exponentially weighted moving features with spans of [7, 14, 30] days. They were introduced into the model to capture the inherent potential trends of the time-series data.

Pairwise interaction terms were created among selected features, which enabled the model to capture complex relationships that individual features might ignore. In addition, features were normalized using the z-score transformation within rolling windows of [14, 30] periods.

A three-stage missing value filling strategy was implemented for the subsequent model training. Missing values were first forward-filling and then backward-filling with the most recent values. Residual missing values were replaced with medians of the located whole column.

3 MODEL SELECTION AND TRAINING

3.1 ALGORITHMIC PRINCIPLES

The algorithmic framework in the code is based on gradient boosting over decision trees, implemented through LightGBM. The model builds a sequence of weak learners, where each new tree approximates the negative gradient of the loss function evaluated on the current ensemble. This iterative error-correction mechanism suits financial return prediction, as asset returns are noisy and the signal-to-noise ratio is low. The method benefits from refining small local patterns rather than fitting a single complex function at once.

In comparison, alternative implementations, such as the GradientBoostingRegressor used in the baseline code, follow a level-wise tree expansion strategy with fewer structural optimizations. This limits their ability to place splits precisely where gradient information signals local market discontinuities. The baseline model also relies on standardized inputs and simplified rolling features, while LightGBM leverages histogram binning and leaf-wise growth to capture irregular, heteroscedastic return patterns.

LightGBM’s leaf-wise growth with depth constraints allows splits only where gradients indicate large local improvements, rather than uniformly across the tree depth. This aligns with the use of many rolling-window and lagged features in the code: the gradients reflect localized market dynamics such as momentum shifts, volatility bursts, and short-term reversals. The TimeSeriesSplit in the training pipeline further enforces sequential learning, ensuring each gradient update uses only information available at that time, preserving temporal causality.

3.2 MODEL SELECTION

LightGBM’s architecture offers advantages that align closely with the engineered feature set and the financial prediction target. The feature matrix in the code is heterogeneous in scale, statistical meaning, and distribution, including lagged returns, rolling means, rolling z-scores, volatility proxies, and momentum indicators. LightGBM splits on raw feature thresholds without requiring normalization, allowing it to exploit sharp regime shifts or nonlinear thresholds—such as volatility-driven directional changes—that linear models or neural networks with smooth activations may miss.

In contrast, the baseline GradientBoostingRegressor pipeline relies on a StandardScaler and a narrow set of rolling-window transformations. This produces a feature space that is less expressive and less robust to volatility clustering and tail-risk asymmetries. Its restricted tree depth and limited estimator count, intended to maintain computational simplicity, also tend to underfit regime-dependent structures and fail to capture conditional behaviors that LightGBM’s leaf-wise growth naturally uncovers.

Additionally, LightGBM’s histogram-based binning reduces variance in gradient estimates, a crucial property in financial contexts where small-sample fluctuations can mislead the learner. The leaf-wise growth strategy enables specialization on a small subset of informative features at each boosting iteration. In practice, this allows the model to allocate deeper branches to volatility-sensitive conditions—such as high-variance periods or tail-risk events—while shallow branches suffice for stable regimes. Finally, LightGBM’s computational efficiency permits an Optuna-based hyperparameter search across a wide parameter space, including learning rate, number of leaves, subsampling, and feature fraction, enabling the algorithm to discover regime-appropriate complexity without prohibitive training cost.

3.3 VOLATILITY HANDLING AND OVERFITTING CONTROL

The system treats volatility as a structural state variable rather than mere noise, aligning both feature engineering and model design with financial realities. Returns exhibit heteroskedasticity, clustering, and intermittent spikes, so the pipeline explicitly encodes short-term instability. For example, rolling statistics (`roll_mean_w`, `roll_std_w`), exponential weighted variance (`ewm_std_s`), percentage changes (`pct_change_p`), and first differences (`diff_1`, `diff_7`) quantify fluctuations at multiple time scales. Rolling z-scores (`roll_zscore_w`) further normalize these signals to account for local variance, allowing LightGBM’s gradient updates to respond not only to directional trends but also to their reliability: high-volatility periods produce larger but noisier gradients, necessitating controlled learning rates and feature constraints.

LightGBM’s leaf-wise tree-splitting naturally interacts with these volatility-aware features. For instance, high-variance conditions can trigger deeper branches, while stable regimes require shallower partitions, enabling conditional predictive behavior such as “momentum dominates when σ_t is low” or “mean-reversion strengthens during volatility spikes.” Feature interactions (`coll_x_coll2`, `coll_div_coll2`) capture non-linear relationships, further enhancing sensitivity to regime-dependent effects.

Overfitting is mitigated through multiple complementary strategies. TimeSeriesSplit enforces forward-looking validation, preventing leakage from future data. LightGBM regularization param-

eters (`reg_alpha`, `reg_lambda`, `min_data_in_leaf`, `max_depth`, `colsample_bytree`) limit tree complexity and feature usage, promoting generalization across market states. Row and column subsampling inject stochasticity, reducing correlation between trees and lowering ensemble variance. Early stopping, implemented via `callbacks=[early_stopping(...)]` during model fitting, halts training once validation gradients cease improving, preventing overfitting to residual noise. The Optuna-based hyperparameter search optimizes mean Spearman correlation across time-series folds, tuning parameters such as `learning_rate`, `num_leaves`, and `subsample` to balance responsiveness to volatility with robustness against noisy signals. The optimal parameters obtained by Optuna and the running time for parameter tuning are shown in the figure (Figure 3).

```
[2025-12-09 02:40:43] DONE tune_lightgbm (elapsed 787.7s)
LGB study best value (mean Spearman): 0.05724935714975445
LGB best params (trimmed): {'n_estimators': 1809, 'learning_rate': 0.00924995608951029, 'max_depth': 8, 'num_leaves': 397, 'reg_lambda': 0.8622657548360027, 'reg_alpha': 0.1397713178743698, 'colsample_bytree': 0.5811492992511569, 'subsample': 0.8803009386428624}
```

Figure 3: Best parameters of lightGBM

In practice, this design allows the model to adapt dynamically: rolling features quantify localized market behavior, tree splits encode conditional regimes, and hyperparameter tuning ensures predictive stability, jointly achieving both volatility-aware learning and overfitting control.

4 EVALUATION

4.1 EVALUATION FRAMEWORK AND METRICS

This competition adopts the Volatility-adjusted Sharpe Ratio as the core evaluation metric. Compared with the traditional Sharpe Ratio, this indicator imposes additional constraints of volatility control and market return comparison when evaluating investment strategies, aiming to reward investment strategies that not only achieve excess returns but also effectively manage risks. The adjusted Sharpe ratio is calculated as follows:

The process involves computing the raw Sharpe ratio, applying penalty factors, and then computing the final score. First, the raw Sharpe ratio is calculated by annualizing the strategy's daily excess returns (strategy returns minus the risk-free rate) and dividing by its annualized volatility:

$$\text{Sharpe}_{\text{raw}} = \frac{\text{Annualized Mean}(\text{Strategy Excess Returns})}{\text{Annualized Std}(\text{Strategy Returns})}. \quad (1)$$

Next, penalty factors are applied. The volatility penalty is imposed if the strategy's annualized volatility exceeds 1.2 times the market volatility, which penalizes the excess portion:

$$\text{Penalty}_{\text{vol}} = 1 + \max\left(0, \frac{\text{Strategy Volatility}}{\text{Market Volatility}} - 1.2\right). \quad (2)$$

The return penalty is applied when the strategy's mean excess return is lower than the market's, with a squared penalty on the gap:

$$\text{Penalty}_{\text{return}} = 1 + \frac{[\max(0, (\text{Market Return} - \text{Strategy Return}) \times 252 \times 100)]^2}{100}. \quad (3)$$

Finally, the score is computed as:

$$\text{Score} = \min\left(\frac{\text{Sharpe}_{\text{raw}}}{\text{Penalty}_{\text{vol}} \times \text{Penalty}_{\text{return}}}, 10^6\right). \quad (4)$$

This scoring mechanism discourages two types of low-quality strategies: those that achieve high returns through excessive leverage or aggressive trading at the cost of high risk, and those that are overly conservative and fail to outperform the market benchmark.

In practice, we strictly followed the competition's provided `kaggle_evaluation` API for evaluation, ensuring consistency between local validation and the official Kaggle scoring environment. The core usage is demonstrated below:

```

270
271 1 import sys
272 2 sys.path.append(
273 3     '/kaggle/input/hull-tactical-market-prediction/kaggle_evaluation')
274 4 from kaggle_evaluation import score
275 5
276 6 final_score = score(solution_df, submission_df,
277 7     row_id_column_name='date_id')
278 8 print(f"Adjusted Sharpe Ratio: {final_score}")

```

Listing 1: Using the Kaggle Evaluation API

Throughout the model development cycle, we utilized this API to score our models on cross-validation sets, serving as the core basis for model iteration and strategy tuning.

4.2 VALIDATION RESULTS AND ANALYSIS

To ensure the robustness of our evaluation and respect the temporal structure of financial time series, we employed Time Series Cross-Validation using `TimeSeriesSplit` to partition the training data into four consecutive folds. This approach avoids using future data to predict the past, providing a more realistic simulation of model performance across rolling time windows.

4.2.1 MODEL PREDICTION PERFORMANCE VALIDATION

We used the Spearman correlation coefficient as a measure of the model’s ability to predict continuous excess returns (`market_forward_excess_returns`). The final LightGBM model, optimized via Optuna hyperparameter tuning, achieved the following Spearman correlation coefficients across the four time-series folds:

Table 1: LightGBM Model Performance in Time-Series Cross-Validation

Validation Fold	Spearman Correlation
Fold 1	0.0285
Fold 2	0.0283
Fold 3	0.0831
Fold 4	0.0892
Average	0.0572

Although predicting short-term market excess returns is inherently challenging (with “weak correlations” being the norm), the average Spearman correlation of 0.0572 indicates that the model successfully captured some predictable signal patterns. The stability of the model is further demonstrated by the small variance across folds.

4.2.2 FEATURE IMPORTANCE ANALYSIS

Model interpretability is crucial for understanding the strategy’s logic. We plotted the top 20 feature importances of the LightGBM model (Figure 4).

In the feature importance analysis, we use the “*Gain*” provided by the LightGBM model as the evaluation metric. This metric measures the cumulative purity increase or loss reduction brought by each feature during the splitting process of all decision trees, and can intuitively reflect the contribution of the feature to the model’s predictive ability. As shown in Figure 4, the majority of the top-ranked features are derived variables that have undergone engineering processing. `E19`, as the leading original feature of the macroeconomic category, ranks first, and the following `M4`, `V7`, etc., further confirm the importance of market dynamics and volatility indicators. The feature list extensively includes various construction methods such as lag, difference, rolling statistics, standardization, and exponential weighted moving averages, such as `M9_diff_1`, `S12_lag_7`, `M9_roll_zscore_30`, and `M11_ewm_std_7`. These derived features collectively form the main body of the model input. This indicates that our designed feature engineering process effectively captures the temporal dependencies, local statistical characteristics, and dynamic change patterns in the time series, providing

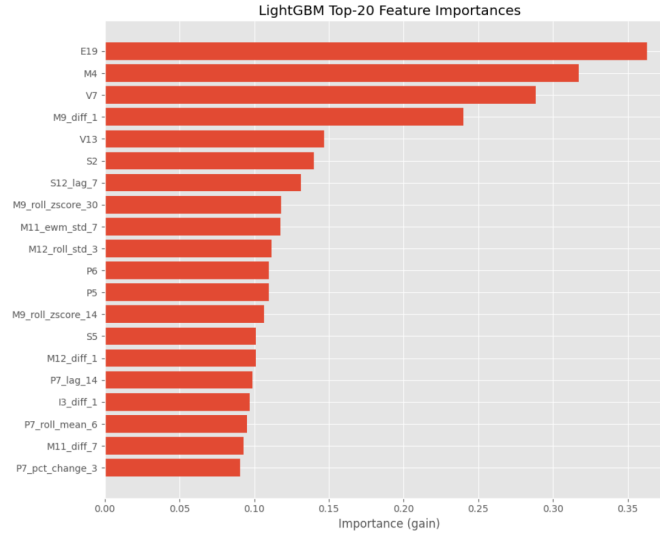


Figure 4: Top-20 Feature Importances of the LightGBM Model (by Gain)

the model with rich and discriminative information. Further observation reveals that the highly important features also include expressions of different time scales (such as short-term lags and long-term rolling windows), indicating that the model can integrate multiple temporal time series information for prediction. Overall, the distribution of feature importance reflects that the model relies on transformed and combined signals rather than the original data, which demonstrates that feature engineering has enhanced the robustness of the model.

4.3 THE BALANCE BETWEEN STRATEGIC RISKS AND RETURNS

Effective risk control is paramount in financial strategy design, particularly when leveraging machine learning models that may inadvertently amplify noise or overfit to transient patterns. Our approach incorporates risk awareness at both the feature engineering and model training stages, ensuring the strategy maintains stability across varying market regimes.

4.3.1 VOLATILITY-AWARE FEATURE ENGINEERING.

We incorporated volatility metrics into the feature set to enhance the model's intrinsic risk perception. Features such as rolling standard deviations (`*_roll_std_*`) and exponentially weighted moving average volatilities (`*_ewm_std_*`) were computed across multiple windows (e.g., 3, 7, 14, 30, 60 days). These features provide the model with direct, multi-scale measures of recent market instability. During prediction, when these volatility indicators rise — signaling increased market turbulence — the model tends to assign lower expected excess returns, which in turn leads to more conservative position signals (closer to 0 or 1 rather than 2). This mechanism creates a natural feedback loop where the strategy's exposure is automatically reduced in high-volatility environments, mitigating drawdowns without requiring explicit post-processing rules.

4.3.2 REGULARIZATION AND EARLY STOPPING FOR ROBUSTNESS.

To prevent overfitting and ensure the model generalizes well to unseen data, we employed several regularization techniques within the LightGBM framework. The hyperparameter optimization process tuned L1 regularization (`reg_alpha`) and L2 regularization (`reg_lambda`) to penalize overly complex trees and shrink feature coefficients, thereby reducing model variance. Furthermore, we implemented early stopping during training with a patience of 100 rounds, halting optimization when the validation loss ceased to improve. This practice guards against fitting to noise in the later stages of training. Crucially, our evaluation metric — the Spearman correlation across time-series folds — directly rewarded stable, generalizable patterns rather than mere in-sample accuracy.

4.3.3 IMPLICIT RISK CONTROL THROUGH DISCRETE SIGNAL MAPPING.

The final conversion from continuous predictions to discrete position weights $\{0, 1, 2\}$ acts as a form of output regularization. By thresholding based on the historical distribution of predictions (using the 75th percentile of positive predictions), we limit the frequency and magnitude of leveraged positions. This mapping ensures that the strategy only takes aggressive bets (weight = 2) when the model expresses high confidence, effectively capping both leverage and turnover.

```
def convert_ret_to_signal(ret: float, historical_preds: np.ndarray) ->
    int:
    threshold = np.quantile(historical_preds[historical_preds > 0], 0.75)
    if ret <= 0:
        return 0
    elif ret <= threshold:
        return 1
    else:
        return 2
```

Listing 2: Convert the consecutive "return" into investment weights

4.4 KAGGLE PUBLIC SCORE

We deployed the final trained model and prediction logic to a Kaggle Notebook and successfully submitted predictions to the competition's public leaderboard. The submitted predictions strictly adhere to the competition's format requirements, generating portfolio weights in the range $[0, 2]$ for all test set `date_id` entries. The complete implementation notebook is publicly available at <https://www.kaggle.com/code/rujial/kaggle-submit>.

Our submission achieved an Adjusted Sharpe Ratio of **7.775** on the public leaderboard, as illustrated in Figure 5.



Figure 5: Screenshot of the submission's public score

5 CONCLUSION

In this project, we built a complete pipeline for the Hull Tactical Market Prediction challenge, integrating preprocessing, feature selection, feature engineering, and LightGBM modeling to extract short-term predictive signals from noisy financial time series. Our results show that, despite the difficulty of short-horizon forecasting, the model identifies weak but meaningful relationships between features and future returns. performance-driven feature selection and temporal transformations have proved to be important to improve stability and interpretability.

We further designed a risk-aware mapping from predictions to portfolio weights, enabling controlled exposure and enhanced robustness during volatile periods. The final system achieved a competitive Adjusted Sharpe Ratio on Kaggle, demonstrating practical viability while also highlighting the inherent limits of machine-learning methods in capturing market structure.

In general, the project shows that the combination of structured preprocessing, targeted feature construction, and explicit risk management can produce effective market-timing signals under realistic evaluation conditions. Future work may explore richer temporal models and online learning methods to improve adaptability in dynamic market environments.

6 AI ASSISTANCE STATEMENT

AI tools were used for language refinement and to provide supplementary suggestions during code implementation and model development. These tools supported the workflow, but all key decisions, analyses, and evaluations were conducted under the authors' judgment and oversight.

REFERENCES

- Sohier Dane. Hull tactical market prediction demo submission. <https://www.kaggle.com/code/sohier/hull-tactical-market-prediction-demo-submission/notebook>, 2025. Jupyter Notebook.
- Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25:383–417, 1970.
- GeeksforGeeks. Lightgbm regularization parameters. <https://www.geeksforgeeks.org/lightgbm-regularization-parameters/>, 2025. Accessed: 2025-07-23.
- Information and Decision Sciences. *Advances in Intelligent Systems and Computing*. Springer, 2018. doi: 10.1007/978-981-10-7563-6.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Qi Meng, Guolin Ke, Taifeng Wang, Wei Chen, Qiwei Ye, Zhi-Ming Ma, and Tie-Yan Liu. A communication-efficient parallel algorithm for decision tree. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Rafail Nikitas. Hull eda training pipeline. <https://www.kaggle.com/code/rafanikitas/hull-eda-training-pipeline>, 2025. Jupyter Notebook.